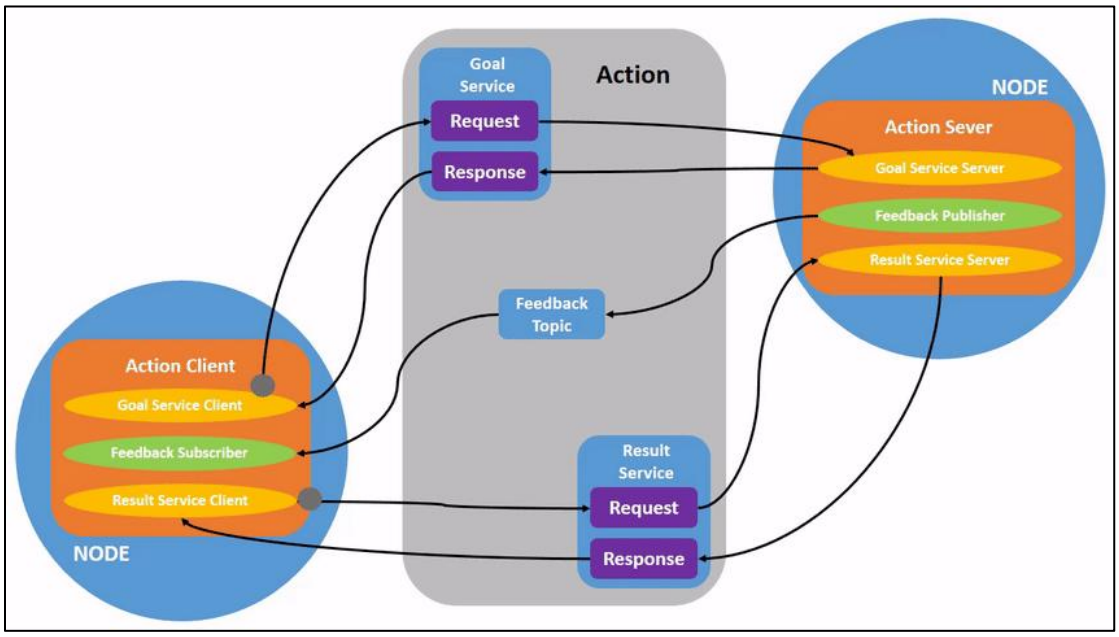
第 10 课 ROS2 动作

1.动作通信简介

动作通信是一种带有连续反馈的通信模型，在通信双方中，客户端发送请求数据到服务端，服务端响应结果给客户端，但是在服务端接收到请求到产生最终响应的过程中，会发送连续的反馈信息到客户端。

动作通信客户端/服务端模型如下：



2.创建接口

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ros2_ws/src/demo_interfaces/**”指令，切换到 demo_interfaces 功能包中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/demo_interfaces/
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$
```

2） 输入“**mkdir action**”指令，创建 action 文件夹。

```
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$ mkdir action
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$
```

3） 输入“**cd action**”指令，进入 action 文件夹。

```
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$ cd action
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces/action$
```

4） 输入指令“**gedit FileDownload.action**”编辑程序，复制下面程序粘贴到文本当汇总，“**ctrl+s**”保存并退出。

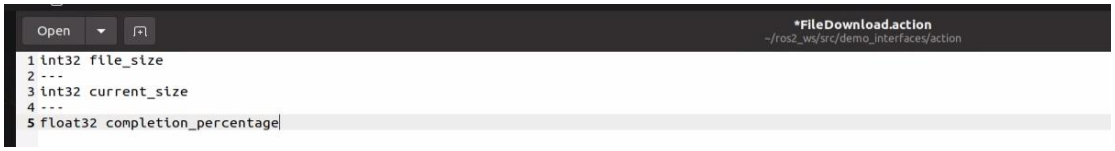
```
int32 file_size

---

int32 current_size

---

float32 completion_percentage
```



5） 先输入“**cd ..**”返回上一级文件夹

```
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces/action$ cd ..
ubuntu@ubuntu:~/ros2_ws/src/demo_interfaces$
```

6） 再输入“**gedit CMakeLists.txt**”，复制下面程序，在下图所示位置添加代码。如需修改，再按下“i”即可修改。修改完成，按下“**Esc**”，输入“**：wq**”保存并退出。

```
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces( ${PROJECT_NAME}

  "msg/Student.msg"

  "srv/AddInts.srv"

  "action/FileDownload.action"
```

```
)
```

```
# the following line skips cpplint (only works in a git repo)
# uncomment the line when this package is not in a git repo
#set(ament_cmake_cpplint_FOUND TRUE)
ament_lint_auto_find_test_dependencies()
endif()


find_package(rosidl_default_generators REQUIRED)
rosidl_generate_interfaces( ${PROJECT_NAME}
  "msg/String.msg"
  "srv/AddInts.srv"
  "action/FileDownload.action"
)
ament_package()
```

3.创建动作通信

3.1 创建服务端

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ros2_ws/src/**”指令，切换到 ros2_ws 工作空间的 src 文件夹中。

```
ubuntu@ubuntu:~$ cd ros2_ws/src/
ubuntu@ubuntu:~/ros2_ws/src$
```

2） 输入“**ros2 pkg create action_demo --build-type ament_python --dependencies rclpy**”并按下回车，创建一个名为“**action_demo**”的功能包，添加依赖关系 rclpy。

```
ubuntu@ubuntu:~/ros2_ws/src$ ros2 pkg create action_demo --build-type ament_python --dependencies rclpy
going to create a new package
package name: action_demo
destination directory: /home/ubuntu/ros2_ws/src
package format: 3
version: 0.0.0
```

3） 输入“**cd action_demo/action_demo/**”指令，切换到 action_demo 功能包下。

```
ubuntu@ubuntu:~/ros2_ws/src$ cd action_demo/action_demo/
ubuntu@ubuntu:~/ros2_ws/src/action_demo/action_demo$
```

4） 输入指令“**gedit action_server.py**”编辑程序，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/action_demo/action_demo$ gedit action_server.py
```

```
import rclpy  # 导入 rclpy 模块
```

```
from rclpy.node import Node  # 导入 Node 类
from rclpy.action import ActionServer  # 导入 ActionServer 类
from demo_interfaces.action import FileDownload  # 导入 FileDownload acti
on 接口
import random  # 导入 random 模块


class FileDownloadActionServer(Node):  # 定义一个继承自 Node 类的 FileDownlo
adActionServer 类
    def __init__(self):
        super().__init__('file_download_action_server')  # 调用父类构造函
数初始化节点
        self._action_server = ActionServer(
            self,
            FileDownload,  # 使用 FileDownload action 接口
            'file_download',  # 定义 action 的名称为 file_download
            self.execute_callback)  # 设置回调函数为 execute_callback


    def execute_callback(self, goal_handle):
        # 服务器端执行的回调函数


        self.get_logger().info(f'Start file download for {goal_handle.req
uest.file_size} bytes...')  # 打印开始下载的日志
        feedback_msg = FileDownload.Feedback()  # 创建 Feedback 消息类型的对
```

```
象


        current_size = 0  # 初始化当前下载的文件大小为 0
        while current_size < goal_handle.request.file_size:
            increment_size = random.randint(1, 10)  # 模拟随机增加下载大小
            current_size += increment_size  # 更新当前下载的文件大小
            if current_size > goal_handle.request.file_size:
                current_size = goal_handle.request.file_size
            completion_percentage = (current_size / goal_handle.request.f
ile_size) * 100  # 计算下载进度百分比


            feedback_msg.completion_percentage = completion_percentage  #
 更新 Feedback 消息的下载进度
            self.get_logger().info(f'Publishing feedback: {completion_per
centage:.2f}% downloaded')  # 打印发布的反馈消息
            goal_handle.publish_feedback(feedback_msg)  # 发布反馈消息
            rclpy.spin_once(self, timeout_sec=1.0)  # 模拟时间的流逝


        goal_handle.succeed()  # 指示目标已完成
        result = FileDownload.Result()  # 创建 Result 消息类型的对象
        result.current_size = current_size  # 设置 Result 消息的当前文件大小
        self.get_logger().info('File download completed!')  # 打印文件下载
完成的日志
```

```
        return result  # 返回结果


def main(args=None):
    rclpy.init(args=args)  # 初始化 ROS 节点
    server = FileDownloadActionServer()  # 创建 FileDownloadActionServer
对象
    rclpy.spin(server)  # 进入主循环
    server.destroy_node()  # 销毁节点
    rclpy.shutdown()  # 关闭 ROS


if __name__ == '__main__':
    main()
```

5) 输入指令“**chmod +x action_server.py**”，并按下回车，为保存的 action_server.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/action_demo/action_demo$ chmod +x action_server.py
```

3.2 创建发布者

1) 输入指令“**gedit action_client.py**”编辑程序，复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
ubuntu@ubuntu:~/ros2_ws/src/action_demo/action_demo$ gedit action_client.py
```

```
import rclpy  # 导入 rclpy 模块
from rclpy.node import Node  # 导入 Node 类
from rclpy.action import ActionClient  # 导入 ActionClient 类
```

```
from demo_interfaces.action import FileDownload  # 导入 FileDownload acti
on 接口


class FileDownloadActionClient(Node):  # 定义一个继承自 Node 类的 FileDownlo
adActionClient 类
    def __init__(self):
        super().__init__('file_download_action_client')  # 调用父类构造函
数初始化节点
        self._action_client = ActionClient(self, FileDownload, 'file_down
load')  # 创建 ActionClient 对象


    def send_goal(self, file_size):
        goal_msg = FileDownload.Goal()  # 创建 Goal 消息类型的对象
        goal_msg.file_size = file_size  # 设置文件大小


        self.get_logger().info(f'Sending file download goal for {file_siz
e} bytes')  # 打印发送目标的日志


        self._action_client.wait_for_server()  # 等待服务器可用
        self.future = self._action_client.send_goal_async(goal_msg, feed
back_callback=self.feedback_callback)  # 发送目标异步请求


        self.future.add_done_callback(self.goal_response_callback)  # 添
加目标响应回调函数
```

```
    def goal_response_callback(self,future):
        goal_handle = future.result()  # 获取目标句柄
        if not goal_handle.accepted:
            self.get_logger().info("Goal rejected")  # 打印目标被拒绝的日志
            return
        self.get_logger().info("Goal accepted")  # 打印目标被接受的日志
        self._get_result_future = goal_handle.get_result_async()  # 获取目标结果异步请求
        self._get_result_future.add_done_callback(self.get_result_callback)  # 添加获取结果的回调函数

    def get_result_callback(self,future):
        self.get_logger().info("File download completed successfully.")  # 打印文件下载成功完成的日志

    def feedback_callback(self, feedback_msg):
        self.get_logger().info(f'Received feedback: {feedback_msg.feedback.completion_percentage:.2f}% downloaded')  # 打印接收到的反馈消息

def main(args=None):
    rclpy.init(args=args)  # 初始化 ROS 节点
    client = FileDownloadActionClient()  # 创建 FileDownloadActionClient
```

```
对象


    result = client.send_goal(100)  # 发送目标


    rclpy.spin(client)  # 进入主循环
    client.destroy_node()  # 销毁节点
    rclpy.shutdown()  # 关闭 ROS


if __name__ == '__main__':
    main()
```

2） 输入指令“**chmod +x action_client.py**”，并按下回车，为保存的 action_client.py 赋予可执行权限。

```
ubuntu@ubuntu:~/ros2_ws/src/action_demo/action_demo$ chmod +x action_client.py
```

4.setup.py 文件设置

setup.py 文件是定义一个 ROS2 包的远数据和构建配置，提供了包的元数据、依赖关系、构建配置和安装逻辑等信息，可以帮助开发者正确构建、安装和使用 ROS2 包。需要将 action_server.py 和 action_client.py 的程序入口写到 setup.py 文件中。

1） 输入“**cd ..**”指令，切换到上一级目录。

```
ubuntu@ubuntu:~/ros2_ws/src/action_demo/action_demo$ cd ..
ubuntu@ubuntu:~/ros2_ws/src/action_demo$
```

2） 输入“**gedit setup.py**”指令，并按下回车，打开 setup.py 文件。

```
ubuntu@ubuntu:~/ros2_ws/src/action_demo$ gedit setup.py
```

复制下面程序,使用快捷键“ctrl+s”保存并退出。

```
'action_server = action_demo.action_server:main',

'action_client = action_demo.action_client:main'
```

```
12          ('share/' + package_name, ['package.xml']),
13      ],
14      install_requires=['setuptools'],
15      zip_safe=True,
16      maintainer='ubuntu',
17      maintainer_email='ubuntu@todo.todo',
18      description='TODO: Package description',
19      license='TODO: License declaration',
20      tests_require=['pytest'],
21      entry_points={
22          'console_scripts': [
23              'action_server = action_demo.action_server:main',
24              'action_client = action_demo.action_client:main'
25          ],
26      },
27  )
```

5.编译运行

1） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的 

，输入指令“**cd ~/ros2_ws/**”指令，切换到工作空间的目录。

```
ubuntu@ubuntu:~$ cd ~/ros2_ws/
ubuntu@ubuntu:~/ros2_ws$
```

2） 输入“**colcon build**”指令，并按下回车，编译工作空间中的功能包。

```
ubuntu@ubuntu:~/ros2_ws$ colcon build
```

3） 再输入指令“**source ./install/setup.bash**”，并按下回车，让环境变量生效。

```
ubuntu@ubuntu:~/ros2_ws$ source ./install/setup.bash
```

4） 输入指令“**ros2 run action_demo action_server**”，并按下回车，启动 action_server 动作服务端。

```
ubuntu@raspberrypi:~$ ros2 run action_demo action_server
```

5） 使用 Nomachine 远程连接工具连接 Jetson Orin Nano 主板，然后双击桌面的



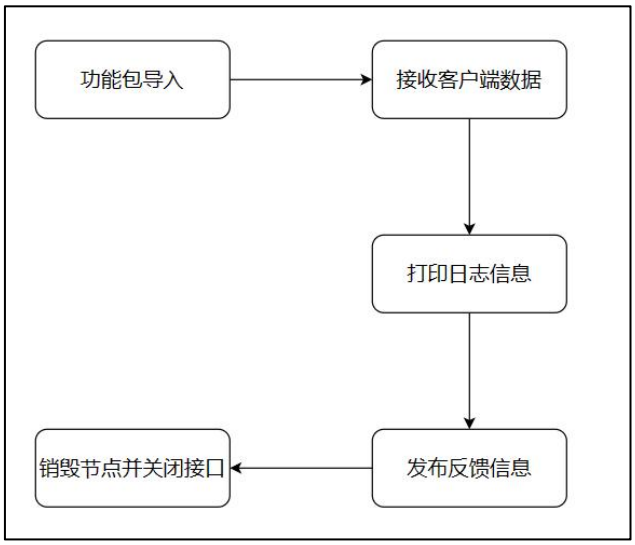
，参考步骤 1）-3）新建一个新的终端窗口并且更新当前的终端环境之后，输入指令：“**ros2 run action_demo action_client**”，并按下回车，启动 action_client 动作客户端，这时候服务端也接收到请求。

```
ubuntu@ubuntu:~/ros2_ws$ ros2 run action_demo action_client
[INFO] [1709030753.332150483] [file_download_action_client]: Sending file downlo
ad goal for 100 bytes
[INFO] [1709030753.336540996] [file_download_action_client]: Goal accepted
[INFO] [1709030753.362044986] [file_download_action_client]: Received feedback:
4.00% downloaded
[INFO] [1709030753.363195472] [file_download_action_client]: Received feedback:
14.00% downloaded
```

6.程序分析

6.1 服务端

根据实现的效果，梳理程序的过程逻辑，如下图所示：



首先定义了一个 FileDownloadActionServer 节点，构造一个 FileDownload 动作类型的 ActionServer 来提供任务服务。ActionServer 的执行回调函数负责模拟执行真实下载任务的流程，它会随机增加下载进度来模拟文件下载，同时通过 goal_handle 对象不断发布下载进度反馈给客户端。任务执行完毕后，会将任务状态设为成功，并返回下载结果。

◆ 主函数

```
def main(args=None):
    rclpy.init(args=args)  # 初始化ROS节点
    server = FileDownloadActionServer()  # 创建FileDownloadActionServer对象
    rclpy.spin(server)  # 进入主循环
    server.destroy_node()  # 销毁节点
    rclpy.shutdown()  # 关闭ROS
```

首先调用 rclpy.init()对 ROS2 Python 接口进行初始化，然后实例化 FileDownloadActionServer()，然后在 ROS2 节点的事件循环执行 server。

◆ FileDownloadActionServer 类

```
class FileDownloadActionServer(Node):  # 定义一个继承自Node类的FileDownloadActionServer类
    def __init__(self):
        super().__init__('file_download_action_server')  # 调用父类构造函数初始化节点
        self._action_server = ActionServer(
            self,
            FileDownload,  # 使用FileDownload action接口
            'file_download',  # 定义action的名称为file_download
            self.execute_callback)  # 设置回调函数为execute_callback

    def execute_callback(self, goal_handle):
        # 服务器端执行的回调函数

        self.get_logger().info(f'Start file download for {goal_handle.request.file_size} bytes...')  # 打印开始下载的日志
        feedback_msg = FileDownload.Feedback()  # 创建Feedback消息类型的对象

        current_size = 0  # 初始化当前下载的文件大小为0
        while current_size < goal_handle.request.file_size:
            increment_size = random.randint(1, 10)  # 模拟随机增加下载大小
            current_size += increment_size  # 更新当前下载的文件大小
            if current_size > goal_handle.request.file_size:
                current_size = goal_handle.request.file_size
            completion_percentage = (current_size / goal_handle.request.file_size) * 100  # 计算下载进度百分比

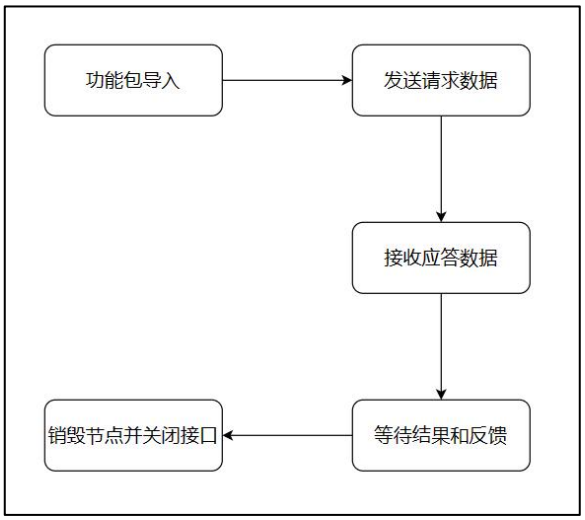
            feedback_msg.completion_percentage = completion_percentage  # 更新Feedback消息的下载进度
            self.get_logger().info(f'Publishing feedback: {completion_percentage:.2f}% downloaded')  # 打印发布的反馈消息
            goal_handle.publish_feedback(feedback_msg)  # 发布反馈消息
            rclpy.spin_once(self, timeout_sec=1.0)  # 模拟时间的流逝

        goal_handle.succeed()  # 指示目标已完成
        result = FileDownload.Result()  # 创建Result消息类型的对象
        result.current_size = current_size  # 设置Result消息的当前文件大小
        self.get_logger().info('File download completed!')  # 打印文件下载完成的日志
        return result  # 返回结果
```

首先创建了一个 ActionServer 对象,将文件下载任务定义为 FileDownload 动作类型,并注册回调函数 execute_callback。这个回调函数在被调用时就会执行文件下载任务,它通过一个 while 循环内部随机增加当前下载量,来模拟文件的实时下载过程。在下载过程中,它不断通过 goal_handle 对象发布当前下载进度作为反馈信息。任务下载完成后,会使用 goal_handle 将任务状态设置为成功,并返回最终下载结果。

6.2 订阅端

根据实现的效果，梳理程序的过程逻辑，如下图所示：



定义了一个 FileDownloadActionClient 类,在构造函数中创建一个 ActionClient 对象,配置文件下载任务类型。客户端提供了一个 send_goal 方法来发送下载目标文件大小作为动作 Goal。同时它还注册了 3 个回调函数,用于处理 Goal 状态响应、任务进度反馈和最后结果。主函数中构造客户端实例,调用 send_goal 发送下载任务,并通过 rclpy.spin()来运行回调函数从而获取任务执行全过程的状态变化。

◆ 主函数

```
def main(args=None):
    rclpy.init(args=args)  # 初始化ROS节点
    client = FileDownloadActionClient()  # 创建FileDownloadActionClient对象

    result = client.send_goal(100)  # 发送目标

    rclpy.spin(client)  # 进入主循环
    client.destroy_node()  # 销毁节点
    rclpy.shutdown()  # 关闭ROS
```

首先调用 rclpy.init()对 ROS2 Python 接口进行初始化，然后实例化 FileDownloadActionClient()，发送一个下载任务，然后在 ROS2 节点的事件循环执行 client。

◆ FileDownloadActionClient 类

```python
class FileDownloadActionClient(Node):  # 定义一个继承自Node类的FileDownloadActionClient类
    def __init__(self):
        super().__init__('file_download_action_client')  # 调用父类构造函数初始化节点
        self._action_client = ActionClient(self, FileDownload, 'file_download')  # 创建ActionClient对象

    def send_goal(self, file_size):
        goal_msg = FileDownload.Goal()  # 创建Goal消息类型的对象
        goal_msg.file_size = file_size  # 设置文件大小

        self.get_logger().info(f'Sending file download goal for {file_size} bytes')  # 打印发送目标的日志

        self._action_client.wait_for_server()  # 等待服务器可用
        self.future = self._action_client.send_goal_async(goal_msg, feedback_callback=self.feedback_callback)  # 发送目标>
异步请求

        self.future.add_done_callback(self.goal_response_callback)  # 添加目标响应回调函数

    def goal_response_callback(self,future):
        goal_handle = future.result()  # 获取目标句柄
        if not goal_handle.accepted:
            self.get_logger().info("Goal rejected")  # 打印目标被拒绝的日志
            return
        self.get_logger().info("Goal accepted")  # 打印目标被接受的日志
        self._get_result_future = goal_handle.get_result_async()  # 获取目标结果异步请求
        self._get_result_future.add_done_callback(self.get_result_callback)  # 添加获取结果的回调函数

    def get_result_callback(self,future):
        self.get_logger().info("File download completed successfully.")  # 打印文件下载成功完成的日志

    def feedback_callback(self, feedback_msg):
        self.get_logger().info(f'Received feedback: {feedback_msg.feedback.completion_percentage:.2f}% downloaded')  # 打>
印接收到的反馈消息
```

创建了一个 ActionClient 对象来与任务服务进行通信。客户端提供了一个 send_goal 方法发送下载目标,并注册了 3 个回调函数用于处理任务不同阶段的状态变化 feedback。send_goal 方法在发送 Goal 后,会通过 future 对象添加回调函数,等待 ActionServer 的响应。这 3 个回调函数分别用来获取 Goal 接收结果、任务进度更新以及完成结果,能覆盖任务整个生命周期。